O Que Agents São Instruídos a Verificar vs. O Que o Consensus Ethereum Discute na Prática: Um WIP Cruzando GitSkills e Web3BlockSet

Pamella Soares
Universidade Estadual do Ceará (UECE)
Fortaleza, Brasil
pamella.soares@aluno.uece.br

***Resumo*—Coding agents dependem de *skills* em linguagem natural para orientar verificação e testes. Em paralelo, o desenvolvimento do consensus Ethereum discute propriedades e falhas concretas em specs e clients. Neste *work-in-progress*, cruzamos dois artefatos empíricos: (i) GitSkills [1]—orientação de teste/verificação em agent skills blockchain; e (ii) Web3BlockSet [2]—Issues/PRs em repositórios de consensus/spec/clients. A provocação é que skills enfatizam verificação *genérica* (unit/coverage e menções a "formal verification"), enquanto a prática do consensus concentra atenção em *fork choice*, *attestation*, *finality* e networking—temas raramente nomeados nas skills. O artigo delimita-se como ideia thought-provoking com evidência preliminar, motivando agentes que traduzem cláusulas de spec em propriedades e testes rastreáveis—agenda alinhada a bounded model checking de specs de consensus [3], [4], sem pretender ainda executar um model checker.**

***Index Terms*—Agent skills, Ethereum consensus, verificação, Mining Software Repositories, work-in-progress**

I. Introdução e Provocação

A especificação de consensus da Ethereum [4] e seus clients (p.ex., Lighthouse, Teku) evoluem sob forte pressão de correção: forks, finality e attestations não são apenas "bugs de produto", mas candidatos a propriedades de segurança/viveza. Ao mesmo tempo, proliferam *agent skills*—instruções em linguagem natural que dizem a coding agents *como* testar e verificar [1]. Surge uma tensão:

> *Se agents são treinados/instrídos sobretudo com verificação genérica (unit tests, coverage, "rode formal verification"), mas a prática do consensus discute componentes específicos da spec, então o elo "cláusula → propriedade → teste/verificação" permanece quebrado.*

Este short WIP coloca essa provocação com dados, sem construir ainda um motor BMC nem estender ESBMC. Nos termos do MSR, é uma ideia interessante/ainda incompleta, com avaliação limitada e caminho explícito para agentes de verificação.

RQ: Como a orientação de teste/verificação em agent skills blockchain (GitSkills) se alinha—ou desalinha—às preocupações observáveis em Issues/PRs de repositórios de consensus/spec/clients (Web3BlockSet)?

a) Por que isto é WIP (e útil).: Não fazemos model checking; não reivindicamos causalidade; não repetimos BcDEx/BaaS [5]. A contribuição é um *mapa de gap* prática×instrução, mais um protocolo para transformar Issues em esboços de propriedade—insumo natural para futuras pipelines agente+BMC.

II. Trabalho Relacionado (posicionamento breve)

BOSE destaca especificidades de engenharia blockchain [6]. Web3BlockSet organiza prática em GitHub com dualidade provider/community e categorias validadas [2]. GitSkills documenta o ecossistema emergente de agent skills [1]. Estudos de BcDEx/BaaS [5] aprofundam desafios de desenvolvedores em plataformas, mas não contrastam *instrução a agents* com *prática de consensus*. Este WIP ocupa esse cruzamento e aponta para a agenda de verificação automatizada de specs [3], [4] como horizonte—não como resultado já obtido.

III. Dados e Método (deliberadamente limitado)

A. Lado prática — Web3BlockSet

Selecionamos repositórios provider ligados a consensus/spec/clients: `ethereum/consensus-specs`, `execution-specs`, `beacon-APIs`, `EIPs`, `annotated-spec`, `sigp/lighthouse`, `ConsenSys/teku`. Sobre Issues/PRs (2018–2025), classificamos lexicalmente preocupações em buckets: `fork_choice`, `finality_justification`, `attestation_aggregation`, `state_transition`, `testing_verification`, `networking_p2p` (demais = `other`).

B. Lado instrução — GitSkills

Usamos a linhagem já filtrada de tipos de teste em skills blockchain (`mandatory_only` → pool de 17.847 skills com $\geq$1 camada de teste) [1], reportando camadas unit/integration/e2e/fork/fuzz_property/formal_symbolic. Complementamos com contagens de termos em skills distintas (`dedup_primary=1`): p.ex. "formal verification", "fork choice", "consensus spec", "beacon chain", "model check".

C. Cruzamento

Comparamos (i) volume de documentos por bucket de prática com (ii) hits de termos/camadas nas skills. A provocação não exige escala idêntica dos eixos—exige desalinhamento qualitativo robusto o bastante para discussão.

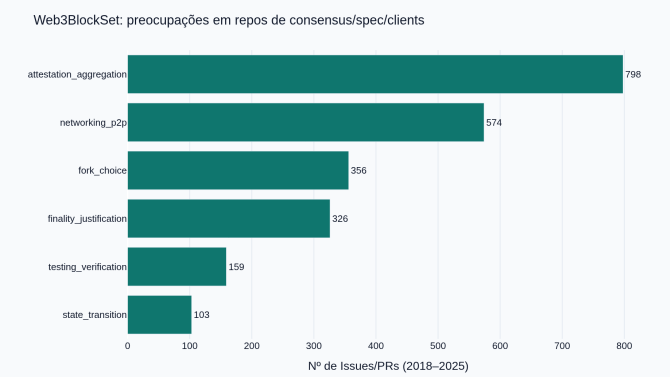


Figura 1. Web3BlockSet: Issues/PRs por preocupação em repos de consensus/spec/clients.

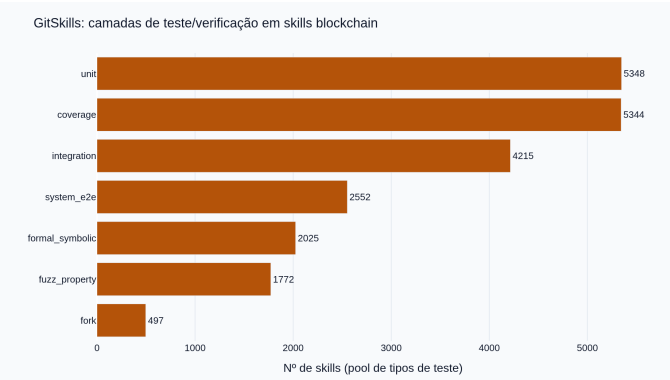


Figura 2. GitSkills: camadas de teste/verificação no pool blockchain.

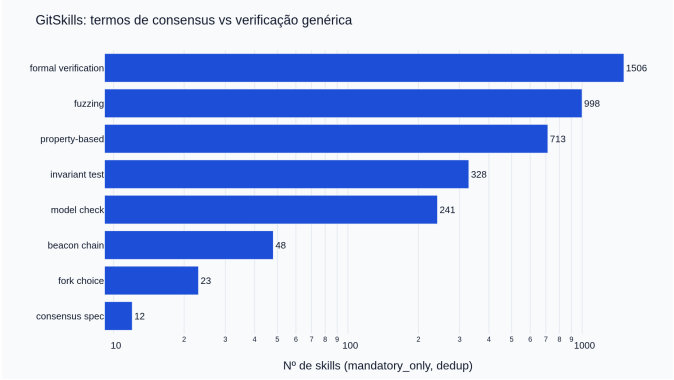


Figura 3. GitSkills: termos de consensus (raros) vs verificação genérica (frequentes). Eixo X em escala log.

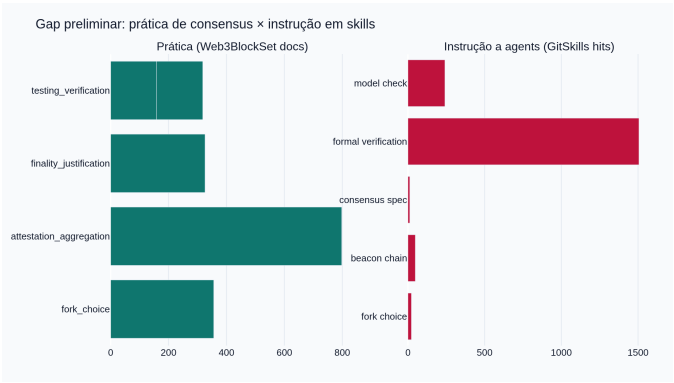


Figura 4. Gap preliminar lado a lado: prática (docs) vs instrução (skills).

D. Validação manual

Amostra cega estratificada por bucket (`data/manual_review/`) pede esboço de propriedade (safety/liveness/reachability) e componente da spec. Protocolo em `PROTOCOL_MANUAL_REVIEW.md`. O objetivo não é relabelar categorias de repositório (já validadas no Web3BlockSet), e sim produzir um **backlog auditável** de propriedades candidatas a partir da prática.

E. O que este WIP conscientemente não faz

Não executa model checker; não mede qualidade semântica das skills além de proxies lexicais; não afirma que agents "falham" em produção—afirma que a *instrução pública* está desalinhada da *ontologia discursiva* do consensus.

IV. Evidência Preliminar

a) Prática (Web3BlockSet).: No recorte (6 repositórios seed; N=10.962 Issues/PRs em 2018–2025), os buckets de domínio dominam as menções explícitas de testing/verification no texto: attestation (798), networking (574), fork choice (356), finality (326), contra apenas 159 documentos no bucket `testing_verification` (Figura 1). Isso *não* implica ausência de testes nos clients; implica que a conversa observável gira em torno de **componentes da spec**.

b) Instrução (GitSkills).: No pool de 17.847 skills com orientação de teste, unit (5.348) e coverage lideram; fork aparece em apenas 497 skills. formal_symbolic (2.025) e fuzz_property (1.772) existem, porém os termos específicos de consensus são escassos nas skills distintas: "fork choice" = 23, "consensus spec" = 12, "beacon chain" = 48, contra "formal verification" = 1.506 (Figuras 2–3).

c) Gap (provocação).: A Figura 4 resume o desalinhamento: a prática nomeia *onde* a correção importa (fork choice, attestations, finality); as skills nomeiam *que* família de técnica usar (formal verification, unit tests), com pouca rastreabilidade a cláusulas do consensus spec. Numericamente, o contraste é gritante na cauda semântica (Tabela I): dezenas de skills citam "fork choice", enquanto centenas de Issues/PRs do recorte discutem o tema; milhares de skills citam "formal verification", enquanto o bucket textual de testing/verification na prática é o menor entre os buckets de domínio. Para um WIP, isso motiva a hipótese central:

> *Agents hoje recebem instruções de verificação desalinhadas da ontologia do consensus Ethereum; fechar esse gap exige agentes que liguem cláusula/Issue → propriedade → oráculo/teste.*

Tabela I
PROXIES DO GAP (UNIDADES DISTINTAS; INTERPRETAÇÃO QUALITATIVA).

Sinal	Prática (docs)	Skills (hits)
fork choice / bucket	356	23
beacon / attestation proxy	798	48
consensus spec term	—	12
testing_verification bucket	159	—
"formal verification"	—	1 506
"model check"	—	241

V. POR QUE ISSO IMPORTA (E COMO CONECTA A BMC/AGENTS)

A. Implicações

- **Para agents de verificação:** tradutores de spec precisam de um backlog empírico de propriedades—Issues/PRs são uma fonte rastreável e versionável.
- **Para BMC / model checking de specs:** propriedades sem âncora na prática viram exercícios; propriedades sem engine viram wishlist. O WIP posiciona a âncora empírica *antes* da engenharia do checker.
- **Para teste gerado por agentes:** suites "traceable to specification clauses" [4] exigem exatamente o elo que o gap sugere estar fraco nas skills atuais.
- **Para pesquisa MSR / Mining Challenge:** demonstra valor em cruzar datasets de *instrução a agents* (GitSkills) com datasets de *prática* (Web3BlockSet) no mesmo domínio crítico.
- **Para formação em FV:** oferece um caminho de entrada sem exigir, no curto prazo, expertise em SMT—mas exige aprender a falar a língua de propriedades (safety/liveness/reachability).

B. Atenções (não overclaim)

Matching lexical é proxy; "formal verification" em skills pode ser shallow; Issues misturam suporte e manutenção; EIPs ampliam o recorte além de consensus puro; GitSkills e Web3BlockSet têm unidades incomparáveis (skills vs docs)—comparamos padrões, não taxas absolutas. A ausência de um termo em skills não prova ausência de conhecimento do autor da skill.

C. Remediações e agenda (4 frentes)

1) **Agora (este WIP):** concluir anotação cega → backlog de propriedades esboçadas com evidência verbatim.
2) **Curto prazo:** agent piloto Issue/cláusula → propriedade em template (safety/liveness) + citação da seção do `consensus-specs`.
3) **Médio prazo:** ligar propriedades a testes existentes do consensus-specs / clients (rastreabilidade).
4) **Longo prazo (fora do short):** traduzir um subconjunto para BMC (p.ex., frontend Python / ESBMC)—aprendizado dirigido alinhado à oportunidade de research assistant, não claim deste artigo.

VI. EXEMPLOS DE ESBOÇOS DE PROPRIEDADE (ILUSTRATIVOS)

Sem validação completa ainda, esboçamos o tipo de saída que a anotação cega deve produzir—útil tanto para o paper quanto para treinar futuros agents:

- **Fork choice / safety:** "o algoritmo de escolha de head não seleciona um bloco que viole regras de justificação/finalização já observadas."
- **Finality / safety:** "dois checkpoints conflitantes não podem ambos ser finalizados."
- **Attestation / validity:** "agregados só incluem attestations com dados consistentes com o estado do epoch."
- **State transition / reachability:** "a partir de um estado pré-slot válido, `process_slots`/`process_block` alcança apenas estados que preservam invariantes de balanço/validadores."

Esses templates são propositalmente informais: o WIP defende que *primeiro* se ancora a propriedade na prática e na spec; *depois* se formaliza para um checker.

VII. POSICIONAMENTO FACE A TRABALHOS E OPORTUNIDADES

Este WIP é ortogonal a estudos de DEx/BaaS [5] e complementar ao plano de tipologias de teste em GitSkills: aqui o foco não é a pirâmide unit→e2e em si, e sim o **descompasso semântico** entre instrução a agents e a ontologia do consensus. Em relação a agendas de bounded model checking para specs Ethereum [3], [4], contribuímos a etapa *upstream*: evidência de quais componentes e falhas a comunidade discute, e quão pouco isso aparece nomeado nas skills. Não reivindicamos implementação de frontend ESBMC neste artigo; reivindicamos um backlog empírico e uma provocação clara para agentes de tradução cláusula→propriedade—precisamente o tipo de ponte empírica que torna produtiva uma colaboração futura com grupos de systems/software security.

VIII. AMEAÇAS À VALIDADE

a) Construto.: Keywords e first-match de buckets são proxies; "formal verification" em skills pode ser menção superficial.

b) Interna.: Um documento pode misturar preocupações; atribuímos um único bucket.

c) Externa.: Seed pequeno de repositórios (sem todos os clients); EIPs alargam o escopo.

d) Confiabilidade.: Validação manual do esboço de propriedades está em curso (n amostral preparado).

IX. CONCLUSÃO

Cruzando GitSkills e Web3BlockSet, este WIP argumenta que a instrução a agents sobre verificação blockchain está **desalinhada** da ontologia do consensus Ethereum discutida na prática: muita "formal verification" genérica nas skills, pouca menção a fork choice/consensus spec; na prática, o contrário em termos de atenção discursiva a componentes. O valor imediato é um mapa de gap e um protocolo para propriedades candidatas; o valor estratégico é preparar a ponte

empírica para agentes e engines de verificação de specs—sem fingir que mining substitui model checking.

DISPONIBILIDADE

Artefatos em `2027_short_paper_bridge`; dados-base Web3BlockSet [2] e GitSkills [1].

REFERÊNCIAS

[1] G. Destefanis, D. Graziotin, M. Vaccargiu, and M. Ortu, "GitSkills: A dataset of agent skills on GitHub," 2026, mSR 2027 Mining Challenge dataset; arXiv:2608.10906.

[2] P. Soares, G. Destefanis, A. C. N. dos Santos, A. A. Araújo, R. Saraiva, and J. Souza, "Web3BlockSet: A dataset for empirical research in blockchain-oriented software engineering," in *Proceedings of the 23rd International Conference on Mining Software Repositories*, 2026.

[3] A. Biere, A. Cimatti, E. Clarke, and Y. Zhu, "Bounded model checking," *Advances in Computers*, 2003, foundational BMC reference.

[4] Ethereum Foundation, "Ethereum consensus specifications," https://github.com/ethereum/consensus-specs, 2024.

[5] P. Soares, G. Destefanis, A. C. N. dos Santos, A. A. Araújo, R. Saraiva, and J. Souza, "Characterizing developer experience in blockchain-as-a-service platforms: An empirical study of GitHub issues and pull requests," *Empirical Software Engineering*, 2025, manuscript.

[6] S. Porru, A. Pinna, M. Marchesi, and R. Tonelli, "Blockchain-oriented software engineering: challenges and new directions," in *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. IEEE, 2017, pp. 169–171.